

A Size-Invariant Deep Reinforcement Learning Policy for the Interval Job Shop Scheduling Problem

Hernán Díaz Rodríguez^{1*}

^{1*}Department of Computing, University of Oviedo, Gijón, Spain.

Corresponding author(s). E-mail(s): diazhernan@uniovi.es;

Abstract

Task durations on a shop floor are seldom known exactly when schedules must be committed. The interval job shop scheduling problem (IJSP) models each processing time as a closed interval; published solvers search each instance separately with metaheuristics. This paper studies the complementary regime: a constructive policy, trained once by deep reinforcement learning, that returns a schedule in seconds. Dimensionless features and a permutation-invariant encoder make its parameter count independent of the numbers of jobs and machines. Trained by proximal policy optimization on four interval Taillard 20×15 instances, it reaches **13.8%** mean relative error against crisp lower bounds on the development instances of that size, against $\approx 46\%$ for the best dispatching rule, and outperforms it zero-shot on every instance of all seven size classes; the same weights transfer to a second benchmark of classical instances. Against an evolved dispatching rule the ordering depends on the inference budget: pass for pass the rule wins, and at **1024** samples each the policy wins 46 of the 70. Three ablations are negative: self-attention does not improve on pooling, multi-size training does not overtake a single-size specialist, and the interval-width features can be removed at no cost, as the closed form of the worst-case makespan predicts. Training on crisp midpoints does cost **0.63** points: what the interval model buys lies in the data rather than the features. A width-penalizing reward traces a tunable width–makespan frontier—yet the entire frontier is reproduced by sample selection from the default policy, without retraining.

Keywords: job shop scheduling, interval uncertainty, deep reinforcement learning, neural combinatorial optimization, size invariance, dispatching

1 Introduction

A production schedule is committed before the work it plans has been done, and the durations it assumes are estimates: tool wear, operator variability and material differences all move them. The job shop scheduling problem, NP-hard (Garey, Johnson, & Sethi, 1976) and industrially ubiquitous, is nonetheless posed in its textbook form as if every processing time were known exactly. Of the formalisms proposed for this uncertainty, stochastic and fuzzy models (González-Rodríguez, Vela, & Puente, 2010) demand distributions or membership functions from the decision maker; the *interval* model asks only for bounds. In the resulting interval JSP (IJSP), each duration is a closed interval, schedule evaluation propagates those intervals, and the makespan is itself an interval, compared through a ranking criterion (Díaz, Palacios, González-Rodríguez, & Vela, 2023a; Lei, 2011).

Makespan minimization in the IJSP is currently the territory of per-instance metaheuristic search: elitist artificial bee colony algorithms (Díaz et al., 2023a; Díaz, Palacios, González-Rodríguez, & Vela, 2023b) reach mean relative errors (RE, measured against crisp Taillard lower bounds) between roughly 6% and 16% depending on the size class. The price is structural: every new instance restarts the search from scratch, for seconds to minutes per run on dedicated hardware.

The deterministic scheduling literature has developed a complementary paradigm, *learning to dispatch*: a deep reinforcement learning (DRL) agent builds the schedule constructively, choosing one eligible operation at a time (Park, Chun, Kim, Kim, & Park, 2021; Tassel, Gebser, & Schekotihin, 2021; Zhang et al., 2020). The training cost is paid once; afterwards the policy returns a schedule for a new instance in seconds, which makes it attractive both as a standalone solver under tight time budgets and as an initializer for search. Yet this paradigm has not crossed over to interval durations: a recent survey of learning-based job shop scheduling (Xu, Mei, Zhang, & Zhang, 2025) lists no method addressing interval-valued processing times.

Two obstacles stand between standard JSP-DRL designs and the IJSP. The first is representational: uncertainty must be visible to the agent, which should treat an operation lasting $[10, 11]$ differently from one lasting $[5, 16]$ even though both share the same midpoint. The second is architectural: common designs tie network dimensions to the instance size, so a policy trained on one size class cannot be *evaluated* on another—at odds with the amortization argument that motivates learning.

We do not reopen the choice of interval ranking. Intervals admit no natural total order, so any comparison of interval makespans embeds a modeling choice; we adopt the lexicographic ranking that a systematic study of the alternatives found most robust for this problem (Díaz, González-Rodríguez, Palacios, Díaz, & Vela, 2020), and keep it fixed throughout so that every method in this paper is judged by the same criterion.

This paper addresses both obstacles and asks three questions. *Q1*: can a constructive policy trained on a handful of interval instances compete with strong hand-crafted constructive baselines? *Q2*: can a single network, trained on one size class, remain useful across the whole benchmark without retraining? *Q3*: which standard ingredients of the JSP-DRL toolbox—attention encoders, multi-size training, automatic hyperparameter configuration—actually pay off at practical training budgets?

Contributions.

1. **A size-invariant, uncertainty-aware policy for the IJSP** (Section 4). All features are dimensionless: temporal quantities are scaled by a per-instance lower bound and interval uncertainty enters as relative widths. A Deep Sets (Zaheer et al., 2017) encoder over the eligible-operation set, combined with a pooled global context, yields policy and value networks of $\approx 1.2 \times 10^5$ parameters, independent of the numbers of jobs and machines.
2. **An empirical characterization** (Section 6) answering Q1–Q3: budget scaling on the training class, zero-shot dominance over the best dispatching rule on all seven Taillard size classes from a single network, and transfer to a second benchmark of classical instances.
3. **A comparison against both families of constructive heuristic** (Section 6.2), on the 70 benchmark instances and with per-instance results reported in full (Appendix A). The hand-crafted side is the traditional dispatching rules—SPT, LPT, MOR, MWKR and EST, plus two Giffler–Thompson variants—which a single greedy pass improves on instance by instance. The learned-symbolic side is a rule evolved by genetic programming (GP) for this same benchmark (Díaz Rodríguez, 2026b), a comparison Xu et al. (2025) single out as missing from the literature: the two paradigms are rarely measured against each other under a common protocol. Here the answer is budget-dependent rather than a verdict: pass for pass the evolved rule wins, and at 1024 samples each the policy does.
4. **An analysis of what earns its cost** (Section 7): a permutation audit of which inputs the network consults, six controlled ablations (inference budget, attention versus pooling, specialist versus multi-size training, and three that delimit what interval awareness contributes) of which four are negative results, a λ -sweep that traces a width–makespan frontier and locates all of it in sample selection rather than in the learned weights, a Monte Carlo comparison of how faithfully the schedules execute, and a two-campaign automatic-configuration study whose racing optima repeatedly failed to transfer to the operating budget.

Section 2 reviews related work and Section 3 states the problem. Section 4 presents the policy, Section 5 the experimental methodology and Section 6 the results; Section 7 analyses them and states the limitations, and Section 8 concludes.

2 Related Work

2.1 Job shop scheduling under interval uncertainty

Uncertain processing times have been modeled with probability distributions, fuzzy numbers and intervals. The fuzzy JSP line is mature (González-Rodríguez et al., 2010; Palacios, González-Rodríguez, Vela, & Puente, 2015); the interval model can be read as its minimal-commitment limit, retaining only support bounds. Lei (2011) opened the interval line with population-based neighborhood search; genetic algorithms with interval rankings (Díaz et al., 2020), robustness-oriented tardiness models (Díaz, Palacios, Díaz, Vela, & González-Rodríguez, 2022), and analytical treatments of special cases (Sotskov, Matsveichuk, & Hatsura, 2020) followed, and interval durations now

appear in richer shop variants as well (Zheng, Xiao, Zhang, & Lv, 2024). On the Taillard-derived benchmark used here, the strongest published results belong to elitist artificial bee colony algorithms (Díaz et al., 2023a, 2023b); these works also fixed the evaluation conventions this paper adopts (interval arithmetic, lexicographic ranking during construction, expected makespan for reporting).

2.2 Priority dispatching rules

The oldest constructive approach to the job shop builds a schedule in a single left-to-right pass, repeatedly choosing which of the eligible operations to place next. A *priority dispatching rule* makes that choice by scoring the candidates with a fixed formula over their attributes. The rules this paper uses as baselines and as verification anchors are the classical single-attribute ones, and we keep their usual acronyms. Two look at the operation itself: *shortest* and *longest processing time* (SPT, LPT) prefer the smallest and the largest duration. One looks at the schedule built so far: *earliest starting time* (EST) prefers the operation that could begin soonest given the current occupation of its machine and the progress of its job. Two look at the job the operation belongs to: *most work remaining* and *most operations remaining* (MWKR, MOR) prefer the job with the most pending work, counted in time units and in operations respectively. Under interval durations these attributes are intervals themselves, and candidates are compared with the interval ranking introduced in Section 3. Surveys catalogue well over a hundred variants of such rules (Haupt, 1989; Panwalkar & Iskander, 1977) and recent ones revisit those still in use (Đurašević & Jakobović, 2018).

A refinement worth separating from the rules themselves is the active-schedule generator of Giffler and Thompson (1960), which at each step restricts the candidates to the conflict set of the machine that would finish earliest, and applies a priority rule only to break ties within that set. The restriction guarantees *active* schedules—no operation can be started earlier without delaying another—and lifts the plain rules substantially; we write G&T-SPT and G&T-MWKR for the two combinations used here.

These rules require one pass and no search, which makes them the cheapest constructive option available. Their limitation is that a fixed formula cannot suit every shop and objective (Haupt, 1989), which motivates the learned rules of Section 2.3, and is also why a constructive policy is compared against these rules and not against per-instance metaheuristics, which belong to a different computational class.

2.3 Learning constructive heuristics for scheduling

Neural combinatorial optimization began with pointer networks (Vinyals, Fortunato, & Jaitly, 2015) and their RL training (Bello, Pham, Le, Norouzi, & Bengio, 2016); attention encoders became standard for routing (Kool, van Hoof, & Welling, 2019; Nazari, Oroojlooy, Snyder, & Takáč, 2018). For the deterministic JSP, Zhang et al. (2020) learn dispatching policies over disjunctive-graph GNN embeddings, Park et al. (2021) refine such representations and report cross-size generalization, and Tassel et al. (2021) contribute an efficient environment formulation. Sampling several rollouts and

keeping the best is a standard inference-time lever (Kool et al., 2019). Permutation-invariant set encoders were formalized as Deep Sets (Zaheer et al., 2017), of which attention is the natural strict generalization; our architecture starts from the simpler construction and evaluates attention as a controlled ablation.

Learning constructive heuristics is older than its neural incarnation: GP hyper-heuristics evolve interpretable priority expressions over hand-crafted attributes (Branke, Nguyen, Pickardt, & Zhang, 2016; Nguyen, Mei, & Zhang, 2017). The survey by Xu et al. (2025) compares the GP and RL families for job shop scheduling and notes the scarcity of comparisons under a common protocol. A companion study (Díaz Rodríguez, 2026b) develops the GP side for the interval benchmark used here, which lets Section 6.2 put the two artifacts on the same instances at matched *inference* budgets. A controlled comparison of the two paradigms with *training* budgets matched as well is beyond the scope of either paper and is the subject of dedicated ongoing work.

3 Problem Statement

Definition 1 (IJSP) An IJSP instance comprises n jobs $J_1, \dots, J_n$ and m machines. Job J_j is a fixed sequence of m operations $(o_{j1}, \dots, o_{jm})$, where o_{jk} requires a specific machine ν_{jk} and an uncertain processing time known to lie in the interval $\mathbf{p}_{jk} = [p_{jk}^L, p_{jk}^U]$, $0 \leq p_{jk}^L \leq p_{jk}^U$. Each machine processes at most one operation at a time and preemption is not allowed.

A solution is a processing order of all nm operations compatible with job precedences; we encode it as a *permutation with repetition* of job indices, where the k -th occurrence of j denotes o_{jk} . Given an order, its *semiactive schedule* starts every operation as early as its job and machine predecessors permit. With component-wise interval addition $[a, b] + [c, d] = [a + c, b + d]$ and join $\max([a, b], [c, d]) = [\max(a, c), \max(b, d)]$, starts and completions are intervals:

$$\mathbf{s}_o = \max(\mathbf{c}_{JP(o)}, \mathbf{c}_{MP(o)}), \quad \mathbf{c}_o = \mathbf{s}_o + \mathbf{p}_o, \quad (1)$$

where $JP(o)$ and $MP(o)$ are the job and machine predecessors of o . The interval makespan collects the job completions component-wise,

$$\mathbf{C}_{\max} = \left[\max_j c_{o_{jm}}^L, \max_j c_{o_{jm}}^U \right], \quad (2)$$

and, because Eq. (1) is monotone in the durations, the executed makespan of *any* realization of the processing times falls inside $\mathbf{C}_{\max}$. Intervals carry no canonical total order, so schedule comparison requires a ranking choice; during construction and search we use the lexicographic order

$$\mathbf{a} \prec_{lex} \mathbf{b} \iff a^U < b^U \vee (a^U = b^U \wedge a^L < b^L), \quad (3)$$

which prioritizes the worst case, in line with min-max robust optimization and with the IJSP literature (Díaz et al., 2020, 2023a). For reporting we follow the field

convention (Díaz et al., 2020, 2023a):

$$\text{RE}(\%) = \frac{E[\mathbf{C}_{\max}] - \text{LB}}{\text{LB}} \times 100, \quad E[\mathbf{C}_{\max}] = \frac{1}{2}(C_{\max}^L + C_{\max}^U), \quad (4)$$

with LB the published lower bound of the crisp Taillard counterpart (Taillard, 1993)—an indicative, hardware-independent yardstick rather than a strict optimality gap, since the crisp JSP and the IJSP are different problems.

Remark 1 For instances whose intervals are symmetric perturbations of a crisp instance, the midpoint scenario coincides with the original crisp durations; comparisons under Eq. (4) are therefore scenario-consistent (midpoint solution quality against the midpoint-scenario bound).

4 Method

Reinforcement learning is learning to decide from experience. An *agent* interacts with an environment in steps: it observes the current *state* s , chooses an *action* a among those available, and receives a numeric *reward* signalling how good the consequences were. The agent’s behavior is its *policy* $\pi(a | s)$: a function that, given the state s , assigns a probability to each available action a . The policy is the object of learning: training increases the probability of actions followed by a high cumulative reward (the *return*).

Policy-gradient methods such as PPO (Schulman, Wolski, Dhariwal, Radford, & Klimov, 2017) judge an action not by its return alone but by comparing that return with what was *expected* from the state where the action was taken; only the surplus—the *advantage*—moves the policy. The expectation is itself learned: a *value function* $V(s)$ estimates the return attainable from state s . In practice both functions share one network with two outputs, a *policy head* that scores the available actions and a *value head* that estimates $V(s)$, trained jointly. This is why the network in this paper has two heads.

Instantiated on scheduling, the agent is a dispatcher. The state is the partial schedule; the available actions are the *eligible* operations—the next unscheduled operation of each job—so the generic $\pi(a | s)$ specializes to $\pi(i | s)$, a distribution over which eligible operation i to schedule next; an *episode* is the construction of one complete schedule, and the reward rests on the makespan achieved. The trained policy is then a dispatching criterion in its own right: at each step it orders the eligible operations by preferences learned from experience rather than by a fixed, hand-written formula.

Figure 1 shows the network, read from the input at the top down to the two outputs. Each eligible operation enters as a row of 16 numbers that describe it: how long it may take, how much work its job still carries, how soon it could start (Section 4.2). A multilayer perceptron (MLP)—the *same* one for every candidate—turns each row into an *embedding*: a vector of h learned quantities that re-describe the candidate.

The number of candidates varies from step to step, so the embeddings are *pooled*: their mean and their componentwise maximum are two fixed-size summaries of the



Fig. 1 The size-invariant policy/value network. Solid edges are the base model’s data path; the grey dashed edge is the per-candidate skip connection; the dashed detour is the self-attention variant of Section 4.4, not part of the base model ($B=0$).

whole set. Joined with 12 numbers describing the shop as a whole (the *global state*), these summaries are condensed by another MLP into a single *context* vector g , a fixed-size summary of the current decision state.

The two heads introduced above read from here—but pooling, by design, discards which candidate is which, so the context alone cannot rank them. The policy head therefore receives two inputs, the shared context g and each candidate’s own embedding, carried past the pooling stage by the *skip connection* visible in Figure 1; it scores every such pair, and a *softmax* turns the scores—the *logits*, in network terms—into the probabilities $\pi(i | s)$ (*masked* means that unused padding slots are excluded). The value head estimates the situation rather than any one candidate, so it reads g alone and outputs $V(s)$. In total, four two-layer MLPs—the candidate encoder ϕ , the context combiner, and the two heads—are composed into a single network and trained jointly end to end.

The dashed detour between the embeddings and the pooling is an *optional* self-attention stage, evaluated as a variant in Section 4.4; the base model, which every headline result of this paper uses, omits it.

Sections 4.1–4.4 formalize, in order, the decision process and its reward, the 16+12 input quantities, the network stages, and the self-attention variant. Training and inference are described with the experimental setting in Section 5.4.

4.1 Dispatching as a Markov decision process

Schedule construction is a finite-horizon *Markov decision process*: the state carries everything relevant to deciding onward, and the horizon is exactly nm steps, one per operation. The state after t steps is the partial schedule—each job’s next-operation pointer, the job and machine completion intervals—and the eligible set $\mathcal{E}_t$, written $\mathcal{E}$ when the step is immaterial, with $1 \leq |\mathcal{E}_t| \leq n$; the action selects one element of $\mathcal{E}_t$, and the transition applies Eq. (1) deterministically.

The *reward* at step t is a weighted sum of six shaping components,

$$r_t = w_{\text{mk}} r_t^{\text{mk}} + w_{\text{pr}} r_t^{\text{pr}} + w_{\text{id}} r_t^{\text{id}} + w_{\text{li}} r_t^{\text{li}} + w_{\text{cr}} r_t^{\text{cr}} + w_{\text{ba}} r_t^{\text{ba}}, \quad (5)$$

each component collapsing intervals by their worst case (upper bound) and scaled by the instance’s lower bound, so that no weight depends on the instance’s magnitude. The weights were fixed by hand during development—they were not part of the automatic-configuration study of Section 5.3, which races optimizer hyperparameters only—and three of them are then adjusted per instance by fixed formulas of the instance’s own statistics; the values below are the effective ones on the 20×15 training instances:

- *terminal makespan* (r_t^{mk} , $w_{\text{mk}}=1.0$): the negative scaled makespan at episode end, plus a bonus when the schedule finishes within 5% of the instance bound—the quantity actually being minimized;
- *job progress* (r_t^{pr} , $w_{\text{pr}}=0.26$): favors the completion of started jobs; its base weight 0.2 is raised by a fixed factor 1.3 whenever durations are intervals;
- *machine idleness* (r_t^{id} , $w_{\text{id}} \approx 0.24$): discourages gaps on machines; grows with the prominence of the instance’s bottleneck machine, from a base of 0.2;
- *local improvement* (r_t^{li} , $w_{\text{li}}=0.15$): the step change of the projected makespan, penalizing deteriorations twice as much as it rewards improvements—a dense signal so early decisions are not credited only nm steps later;
- *operation criticality* (r_t^{cr} , $w_{\text{cr}}=0.1$): favors operations on the emerging critical path, a standard proxy for a compact schedule;
- *load balance* (r_t^{ba} , $w_{\text{ba}}=0.1$, its cap, reached whenever machine loads vary as much as here): discourages lopsided machine loads.

The reward shapes training only; every reported result is measured by the RE of Eq. (4).

4.2 Dimensionless, uncertainty-aware features

The representation expresses every temporal quantity as a fraction of the instance’s lower bound LB (its worst case if the bound is itself an interval), and every uncertainty as a ratio of interval width to magnitude. Absolute times are not comparable across

Table 1 Per-operation features (all dimensionless). $d = \mathbf{p}_{jk}$, $\mathbf{est}$ = earliest start interval, $\mathbf{rem}$ = remaining work interval of the job after o ; $\text{mid}(\cdot)$ is the interval midpoint; $L(\mu)$ the remaining worst-case load of machine μ ; $\bar{L}$ its average over machines; $\mathbf{c}_J$ and $\mathbf{c}_\mu$ the completion intervals of the operation’s job and machine over what is scheduled so far.

#	Feature	Definition
1–2	duration bounds	$d^L/\text{LB}, d^U/\text{LB}$
3	duration uncertainty	$(d^U - d^L)/\text{mid}(d)$
4–5	earliest start bounds	$\text{est}^L/\text{LB}, \text{est}^U/\text{LB}$
6	start uncertainty	$(\text{est}^U - \text{est}^L)/\text{mid}(\mathbf{est})$
7–8	remaining job work	$\text{rem}^L/\text{LB}, \text{rem}^U/\text{LB}$
9–10	job position	$(m - k - 1)/m, k/m$
11	machine load	$L(\mu)/\text{LB}$
12	potential idle gap	$\max(0, c_J^U - c_\mu^U)/\text{LB}$
13	machine congestion	$L(\mu)/\bar{L}$
14	slack	$(\text{est}^U - \min_{o' \in \mathcal{E}} \text{est}_{o'}^U)/\text{LB}$
15–16	completions	$c_\mu^U/\text{LB}, c_J^U/\text{LB}$

Table 2 Global state features (all dimensionless aggregates, notation as in Table 1; no dimension depends on n or m). Rows that list several statistics contribute one feature each, for 12 in total.

#	Feature	Definition
1	progress	scheduled operations / nm
2	partial makespan	$\max_\mu c_\mu^U/\text{LB}$
3–4	machine completions	mean and std of c_μ^U/LB
5–7	remaining loads	mean, max and std of $L(\mu)/\text{LB}$
8–9	remaining job work	mean and max of job remaining / LB
10	eligibility	$ \mathcal{E} /n$
11	pending uncertainty	$\sum (\text{widths}) / \sum (\text{midpoints})$ of unscheduled ops
12	total load	$\sum_\mu L(\mu) / (\text{LB} \cdot m)$

instances, since their magnitude reflects the time scale of the instance at hand; a policy trained on them would be tied to the scale of its training set. Under the dimensionless representation, what the policy observes is comparable across instances and across sizes, which is one of the two components of size invariance—the other being the architecture of Section 4.3. Each eligible operation $o = o_{jk}$ on machine $\mu = \nu_{jk}$ is described by the 16 dimensionless quantities of Table 1; the global state is the 12-dimensional aggregate of Table 2.

Every quantity consulted by the dispatching rules of Section 2.2—processing times, remaining work, earliest start times, slack, machine load—appears among the per-operation features, so that no advantage over those rules follows from a wider per-operation vocabulary. Two groups of features extend this shared vocabulary: the interval dimension (bounds and relative widths of every temporal quantity, making

the uncertainty explicit) and the global state (aggregates over the whole shop, which no per-operation priority rule consults). The audit of Section 7.1 reports, a posteriori, which of these inputs the trained network actually uses. Two properties of the representation are supported by the ablations of Section 7.4: interval widths enter as relative, hence non-linear, magnitudes, so they are not linear combinations of the remaining inputs; and the normalization is set jointly with the initialization and the learning rates.

4.3 Size-invariant architecture

The remaining obstacle is that the number of candidates changes at every step and from instance to instance, while a neural network requires inputs of fixed dimension. The architecture handles this with two elements: the *same* network reads every candidate, so any number of them can be processed; and the resulting embeddings are *pooled* into fixed-size summaries, so the scoring stage can see the whole decision at once. Formally, let $x_i \in \mathbb{R}^{16}$ be the features of eligible operation i and $g_0 \in \mathbb{R}^{12}$ the global state; the construction has three stages (Figure 1).

Embed. The shared two-layer MLP ϕ (width $h=128$, LayerNorm, ReLU, orthogonal initialization) maps each candidate to an embedding $\phi_i = \phi(x_i)$. The whole eligible set is processed in one forward pass, as the $|\mathcal{E}| \times 16$ matrix of its rows; sharing the weights across rows is what lets the layer accept any number of candidates, and makes this stage invariant to the number of jobs. Candidates do not interact here: ϕ_i depends on x_i alone.

Pool into a context. The variable-size set $\{\phi_1, \dots, \phi_{|\mathcal{E}|}\}$ must be summarized into a fixed-size vector before anything global can be computed. Mean and max pooling provide two complementary permutation-invariant summaries (Zaheer et al., 2017) which, concatenated with the global state g_0 , are mixed into the context:

$$g = \text{MLP}_{ctx}([\frac{1}{|\mathcal{E}|} \sum_i \phi_i; \max_i \phi_i; g_0]) \in \mathbb{R}^h. \quad (6)$$

The stage has no parameters of its own: it maps the $|\mathcal{E}| \times h$ embeddings to the $2h$ summaries the context MLP consumes. Both statistics are taken over the real candidates only—in batched training the sets are padded to the batch maximum, and the mean divides by the true count while the max ignores the padded rows. This embed-then-pool pattern is the *Deep Sets* construction of Zaheer et al. (2017), whose central result is that *any* function invariant to the order of a set can be written in this form—so this construction entails no loss of expressiveness in principle. The result tables refer to the base model by this name, in contrast to its attention variant (Section 4.4).

Score. The *policy head* rates each pair (candidate embedding, context)—“how good is operation i given the decision as a whole”; the *value head* computes the $V(s)$ of the advantage estimation, reading the context alone, since expectations concern the situation, not any one candidate:

$$\pi(i | s) = \text{softmax}_i \text{MLP}_\pi([\phi_i; g]), \quad V(s) = \text{MLP}_V(g). \quad (7)$$

Table 3 specifies the four trainable blocks. Each is a two-layer perceptron with Layer-Norm and ReLU between its layers and a linear output; all hidden layers are initialized orthogonally with gain $\sqrt{2}$ and zero biases. The output layer of the policy head uses gain 10^{-2} , so at the start every candidate receives a nearly equal score and the untrained policy is close to uniform; the output layer of the value head uses gain 1. Pooling and the masked softmax carry no parameters, so those four blocks account for the whole network: 120,322 parameters, no dimension of which depends on n or m .

Table 3 Layer specification of the policy/value network at the hidden width $h=128$. Shapes are input, hidden and output dimensions.

Block	Shape	Parameters
Candidate encoder ϕ	$16 \rightarrow 128 \rightarrow 128$	18,944
Pooling (mean, max)	$ \mathcal{E} \times 128 \rightarrow 256$	—
Context MLP	$268 \rightarrow 128 \rightarrow 128$	51,200
Policy head	$256 \rightarrow 128 \rightarrow 1$	33,281
Value head	$128 \rightarrow 128 \rightarrow 1$	16,897
Base network ($B=0$)		120,322
Attention block (each)	128, 4 heads of 32; feed-forward $128 \rightarrow 256 \rightarrow 128$	132,480

4.4 The attention variant

Pooling summarizes the candidate set by two statistics, and every candidate is then scored against that same summary. What this cannot express is a comparison between two *particular* candidates: whether operation i should take the machine now may depend on which other eligible operation would occupy it next, and the mean embedding does not identify that candidate. The embeddings of all $|\mathcal{E}|$ candidates are available at the decision step, so the missing operation is not information but a path between them. *Self-attention* (Vaswani et al., 2017) is the standard remedy, and the component most commonly adopted in the neural combinatorial optimization literature (Section 2.3). In an attention layer every candidate emits a *query*, a *key* and a *value*; the similarity between one candidate’s query and another’s key decides how much of that other’s value is added into the first’s embedding. Each candidate is thereby rewritten as a content-weighted reading of all the others, with the weights computed from the embeddings themselves rather than fixed in advance.

We build this as a variant to be tested, not as part of the proposed model. A stack of B pre-layer-normalization (pre-LN) transformer blocks transforms the embeddings after the shared encoder and before pooling, that is, between the embedding and pooling stages of Figure 1. Writing $\Phi \in \mathbb{R}^{|\mathcal{E}| \times h}$ for the stacked embeddings, each block computes

$$Z = \Phi + \text{MHA}(\text{LN}(\Phi)), \quad \Phi' = Z + \text{FFN}(\text{LN}(Z)), \quad (8)$$

where MHA is multi-head scaled dot-product attention (4 heads of dimension 32, no dropout), FFN is a two-layer perceptron ($128 \rightarrow 256 \rightarrow 128$, ReLU) applied to

each row, and LN is LayerNorm. Both additions are *residual*—the block adds to its input instead of replacing it—so an untrained block is close to the identity and the stack starts near the base model, the property for which the pre-LN arrangement is preferred over the original post-LN one (Xiong et al., 2020). No positional encoding is used: without one, self-attention is *permutation-equivariant*—reordering the candidates reorders the outputs correspondingly—so the pooled context remains permutation-invariant and the guarantees of Section 4.3 are preserved. Padding slots are masked out of the attention weights.

Attention is a flag on the same code path, and $B=0$ reproduces the base network exactly—which is what makes the comparison of Section 7.3 a controlled one, differing in this component and nothing else. A single block carries 132,480 parameters—the query, key and value projections, the output projection, two layer normalizations and the feed-forward network (Table 3)—so the $B=2$ variant we evaluate grows the network from 1.2×10^5 to 3.9×10^5 parameters, a factor of 3.2, and makes an episode about 30% more expensive.

During batched training, variable-size candidate sets are padded to the batch maximum with a boolean mask applied to attention, pooling and logits; unit tests verify that masked-padded forward passes are numerically identical to unpadded ones.

5 Experimental Methodology

5.1 Instances, data split and metric

Experiments use the 70 interval Taillard instances TA1–TA70 that anchor the recent IJSP literature (Díaz et al., 2023a, 2023b): seven size classes from 15×15 to 50×20 , each crisp duration replaced by an integer interval symmetric around it, with half-width drawn uniformly up to 15% of the original value (Díaz et al., 2020). The benchmark is openly available (Díaz Rodríguez, 2026a). Of the 70 instances, four carry gradients: the final model trains on TA11–TA14 (20×15) only. Six more of the same class, TA15–TA20, form the *development set*—never used for gradients, but every design and configuration decision was judged on their performance, so they are reported separately and excluded from claims about unseen data. The remaining 60 instances were touched by neither training nor development. A second benchmark—interval versions of twelve classical instances (FT10, FT20, La21, La24, La25, La27, La29, La38, La40 and ABZ7–9), generated with the same symmetric scheme (Díaz et al., 2020)—is reserved for the cross-family evaluation of Section 6.3, and plays no part in training or development either.

5.2 Computational environment

Table 4 lists the machine and the software stack. One remark matters for reading the costs reported later: every number in this paper—training, evaluation, the dispatching-rule baselines and the ablations—was produced on the CPU. The network is small enough (Section 4.3) that a forward pass is negligible next to the environment step that follows it, so the GPU buys nothing for a single rollout; it was used only for the vectorized batched rollouts of the operating-fidelity tuning campaign (Section 5.3),

Table 4 Machine and software used for every experiment in this paper.

Component	Specification
CPU	AMD Ryzen 5 3400G, 4 cores / 8 threads, 3.7 GHz
Memory	32 GB
GPU	NVIDIA GeForce RTX 5060 Ti, 16 GB (tuning only)
Operating system	Windows 10 Pro 22H2 (64-bit)
Language	Python 3.12.6
Learning	PyTorch 2.9.1, CUDA 13.0 build
Numerics	NumPy 2.3.5, SciPy 1.17.0
Figures	Matplotlib 3.10.8; svglib 2.1.0 (diagram)
Tuning	irace 4.4.3 on R 4.6.1 (Section 5.3)

Table 5 Hyperparameters, identical across every experiment reported in this paper.

Hyperparameter	Value	Hyperparameter	Value
<i>Optimizer</i>		<i>PPO objective</i>	
learning rate (Adam)	$3 \cdot 10^{-4}$	clipping range	0.2
Adam ϵ	10^{-5}	GAE λ	0.95
gradient clipping	0.5	discount factor γ	1.0
<i>Network and schedule</i>		<i>PPO updates</i>	
hidden width h	128	epochs per update	4
weight initialization	orthogonal	minibatch size	256
policy update	every 4 ep.	entropy coefficient	0.01
greedy evaluation	every 10 ep.	advantage normalization	per update

where many episodes advance in lockstep and the batched forward pass does pay. The method therefore carries no dependence on specialized hardware.

5.3 Hyperparameter configuration

All reported results use the hyperparameters of Table 5 *without problem-specific tuning*, and none of its values was tuned for this problem. To assess whether tuning would improve on them, we ran two automatic configuration studies with irace (López-Ibáñez, Dubois-Lacoste, Pérez Cáceres, Birattari, & Stützle, 2016). The first raced 330 experiments (309 used) over the 8-parameter space of Table 6 at reduced fidelity: one training instance, 300 episodes, ~ 8 minutes per evaluation. Its elites agreed on a more aggressive update regime and kept minibatch size and update period at their defaults, but at the full 1000-episode operating budget the best of them scored 14.5% mean RE against 13.4% for the defaults, at $\sim 60\%$ higher training cost: the low-fidelity optimum did not transfer.

The second campaign raced 295 experiments at operating fidelity—1000 episodes on two instances per evaluation—over the reduced 6-parameter space of Table 6,

Table 6 Search space of the two irace campaigns. Real parameters were sampled on a logarithmic scale, the rest from the listed values.

Parameter	Default	Campaign 1 (330 exp.)	Campaign 2 (300 exp.)
learning rate	$3 \cdot 10^{-4}$	$[10^{-4}, 10^{-3}]$	$[10^{-4}, 10^{-3}]$
entropy coefficient	0.01	$[0.003, 0.03]$	$[0.003, 0.03]$
clipping range	0.2	0.1, 0.2, 0.3	0.1, 0.2, 0.3
epochs per update	4	2, 4, 8	4, 8
GAE λ	0.95	0.90, 0.95, 0.98	0.90, 0.95, 0.98
hidden width	128	64, 128, 256	128, 256
minibatch size	256	128, 256, 512	fixed
update period	4 ep.	2, 4, 8 ep.	fixed

seeded with the defaults and the first campaign’s elites. It converged to a region distinct from the defaults and eliminated the seeded default during racing; yet its winner, retrained on the four training instances over three seeds, again failed to improve: 14.73% against 13.52% over the six development instances. Automatic configuration therefore does not beat the defaults of Table 5 for this problem *even when performed at the operating budget*, so the reported results do not depend on fine tuning. In both campaigns the racing optimum differed from the confirmation optimum, which matters for automatic DRL configuration: single-run training outcomes are too noisy to separate near-equivalent configurations.

5.4 Training and inference

The policy is trained with PPO (Schulman et al., 2017) under the hyperparameters of Table 5, most of which are standard PPO values. Training on a set of instances proceeds in blocks, one instance at a time, with the weights carried over from each block to the next. A *rollout* is one complete construction episode played by the current policy: *sampled* when each operation is drawn from π , as during training, and *greedy* when every step takes the most probable operation. Every tenth episode (Table 5), the current policy is additionally evaluated with one greedy rollout. Whenever an episode of either kind improves on the best makespan found so far, its solution and a copy of the weights that produced it are stored; the stored weights are the *checkpoint* used at evaluation time.

The discount factor is $\gamma = 1$ rather than the 0.99 customary in deep RL (Schulman et al., 2017). The horizon here is finite and known, so future rewards can be credited in full; at $\gamma = 0.99$ the terminal makespan—the quantity being minimized—would reach the earliest decisions of a 20×15 instance attenuated to $0.99^{nm} \approx 0.05$ of itself. Updates are likewise applied over transitions pooled from four consecutive episodes rather than from one, in minibatches of 256 and with advantages normalized within each update, because the nm steps of a single episode are few and strongly correlated: pooling widens the sample the advantage estimate is computed from, and normalizing keeps the gradient scale comparable across instances of different size.

Algorithm 1 Best-of- N inference

```
1:  $(\sigma^*, \mathbf{C}_{\max}^*) \leftarrow$  greedy rollout of  $\pi$ 
2: for  $i = 1$  to  $N - 1$  do
3:    $(\sigma, \mathbf{C}_{\max}) \leftarrow$  sampled rollout of  $\pi$ 
4:   if  $\mathbf{C}_{\max} \prec_{lex} \mathbf{C}_{\max}^*$  then  $(\sigma^*, \mathbf{C}_{\max}^*) \leftarrow (\sigma, \mathbf{C}_{\max})$ 
5:   end if
6: end for
7: return  $\sigma^*$ 
```

Best-of- N inference.

At evaluation time we exploit the stochastic policy: one greedy rollout plus $N - 1$ sampled ones, keeping the best solution under the ranking of Eq. (3) (Algorithm 1). The deployed budget is $N=64$; results are additionally reported for a single greedy pass and, in the budget-matched comparisons of Sections 6.1 and 6.2, for $N=1024$. This choice is motivated empirically in Section 7.2: during training, the best solutions frequently originate from sampled episodes, i.e. the policy is more valuable as a *distribution* than as its argmax—consistent with sampling-based decoding in neural combinatorial optimization (Kool et al., 2019).

5.5 Baselines

Constructive baselines are the dispatching rules of Section 2.2—SPT, LPT, MOR, MWKR and EST—and the two Giffler–Thompson combinations. The results of Section 6 carry the strongest of each family—MOR, EST and G&T-MWKR—while SPT, LPT, MWKR and the SPT tie-break of Giffler–Thompson are dominated by a wide margin in every size class and are not reported further. The learned-symbolic baseline is the dispatching rule evolved by genetic programming for this benchmark in the companion study (Díaz Rodríguez, 2026b), compared budget by budget in Section 6.2.

Three published IJSP metaheuristics are included as reference yardsticks rather than as direct competitors, since each searches every instance separately for minutes on dedicated hardware and reports 30-run averages: the genetic algorithm (GA) of Díaz et al. (2020), the enhanced swarm artificial bee colony ESABC (Díaz et al., 2023a) and the filtered elitist artificial bee colony fEABC (Díaz et al., 2023b), the strongest of the three. Their figures are quoted from those papers, not re-run here.

6 Results

6.1 In-size performance and budget scaling

Table 7 reports per-instance RE on the development set for increasing training budgets over ten independent training runs, and Figure 2 the aggregate picture. Three observations: (i) the policy scales monotonically with budget ($27.4\% \rightarrow 17.4\% \rightarrow 13.8\%$ mean RE at 100/300/1000 episodes per instance), with the return per tripling of the budget falling from 10.0 points to 3.6; (ii) the spread between training runs collapses

Table 7 RE (%) per development instance, mean [best] over ten training runs differing only in their random seed, Deep Sets model, best-of-64 inference. The rightmost column is the standard deviation across the ten per-seed means over the six instances; the last row is the strongest constructive baseline, a single deterministic pass.

Budget	TA15	TA16	TA17	TA18	TA19	TA20	Mean	SD
100 eps	30.2 [21.9]	26.3 [19.7]	26.8 [18.1]	26.7 [18.4]	28.7 [21.5]	25.8 [18.5]	27.4	4.69
300 eps	19.0 [16.3]	16.3 [14.3]	16.6 [13.3]	18.5 [16.2]	17.2 [13.3]	16.6 [14.9]	17.4	1.30
1000 eps	14.9 [12.1]	13.2 [11.8]	12.9 [10.4]	16.3 [14.6]	12.8 [11.1]	12.8 [10.5]	13.8	0.47
G&T-MWKR	30.6	34.0	17.7	31.8	29.3	24.1	27.9	—

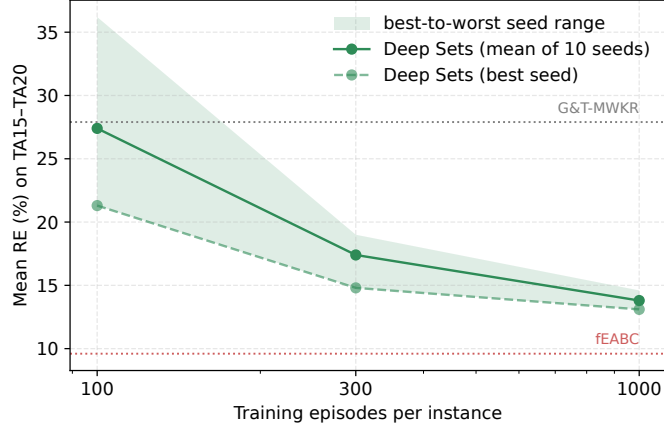


Fig. 2 Mean RE on the development set against the *training* budget, the episodes spent once per instance before deployment, on a logarithmic axis and over ten training runs. The band spans the best and worst of them; the dotted lines are G&T-MWKR and fEABC on the same six instances.

as the budget grows, the standard deviation across the ten seed means falling from 4.69 points at 100 episodes to 1.30 at 300 and 0.47 at 1000—a factor of ten—so what the extra budget buys is not only a better mean but a reproducible one; (iii) the gap to the strongest constructive baseline is a factor ≈ 2.0 . Figure 2 places the curve between two dotted references measured on the same six instances: G&T-MWKR above, at 27.9%, and fEABC below, at 9.6%, the latter a yardstick from a different computational class.

6.2 Zero-shot cross-size generalization and position among constructive methods

The size-invariant design allows direct evaluation of the 20×15 -trained network on any class. Evaluation is *zero-shot*: the trained weights are applied to sizes they never saw, with no additional training or fine-tuning of any kind. Table 8 reports RE over all seventy instances by size class, with its per-instance values in the appendix; the

Table 8 Mean RE (%) per size class over the 70 interval Taillard instances, ten per class. The 20×15 column (†) is the training and development class.

Method	15×15	20×15 [†]	20×20	30×15	30×20	50×15	50×20	All
EST	32.3	49.2	41.5	42.4	56.6	33.2	41.0	42.3
MOR	41.4	46.7	47.3	44.1	58.8	34.1	45.9	45.5
G&T-MWKR	26.7	29.0	30.9	33.6	36.8	23.9	25.5	29.5
GP rule, one pass	15.7	16.6	18.1	21.1	24.1	13.8	14.6	17.7
Policy , greedy	16.9	18.3	18.5	21.2	26.1	15.8	18.8	19.4
GP rule, 64 samples	12.6	15.9	16.8	17.4	23.4	11.7	13.4	15.9
Policy , 64 samples	11.0	13.6	14.8	16.8	22.0	12.2	15.5	15.1
GP rule, 1024 samples	10.2	14.0	15.1	15.5	21.0	10.6	12.1	14.1
Policy , 1024 samples	8.8	11.2	12.9	15.0	19.6	10.4	13.4	13.0
fEABC (30 runs)	5.9	8.7	10.3	9.8	16.4	5.7	9.0	9.4

evolved rule of the companion study (Díaz Rodríguez, 2026b) appears beside the policy at each budget, deterministic at one pass and randomized by ϵ -greedy dispatching when sampled, and fEABC closes the table as a 30-run average from a different computational class. Figure 4 shows the two learners’ per-instance distributions within each class at the paired budgets.

Read zero-shot, every seed’s network at 64 samples outperforms G&T-MWKR, the strongest constructive baseline, on *every* one of the seventy—210 instance—seed pairs without an exception—with graceful degradation (class means from 11.0% on 15×15 to 22.0% on the hardest transfer, 30×20) and no failure mode up to 1000-operation episodes. Degradation is not monotone in instance size: 50×15, at 12.2%, transfers better than either 30-job class, which the dispatching rules also find easier, so the difficulty the network inherits is the instances’ and not a limit of the transfer.

Against the GP rule the comparison is not one-sided: its outcome depends on the inference budget both methods receive (Figure 3). With one pass each, the rule is better. It averages 17.7% against the policy’s 19.4%, and the policy improves on it in only 21 of the 70 instances (median deficit 2.0 points, $p = 4.6 \times 10^{-4}$). This comparison is asymmetric, however: the 17.7% rule is the best of 30 independent evolutions, selected on these same 70 instances, while the 19.4% is a mean over the policy’s three seeds. Against the mean of the 30 evolutions, 18.99%, the deficit at one pass is 0.4 points. The selected rule is nonetheless kept as the comparison, because it is the published artifact and the harder test. With 1024 samples each—the GP rule randomized by the ϵ -greedy dispatching its study provides for this purpose—the ordering reverses: 13.0% against 14.1%, with the policy better on 46 of the 70 (median advantage 1.3 points, $p = 1.4 \times 10^{-3}$).

In conclusion, the network’s advantage lies in the distribution, whose samples improve faster with budget than the randomized rule’s. The best-of- N ablation of Section 7.2 reaches the same conclusion from the training side. The policy’s standalone RE (13.8% in-size, 11–22% zero-shot class means) places it between the dispatching

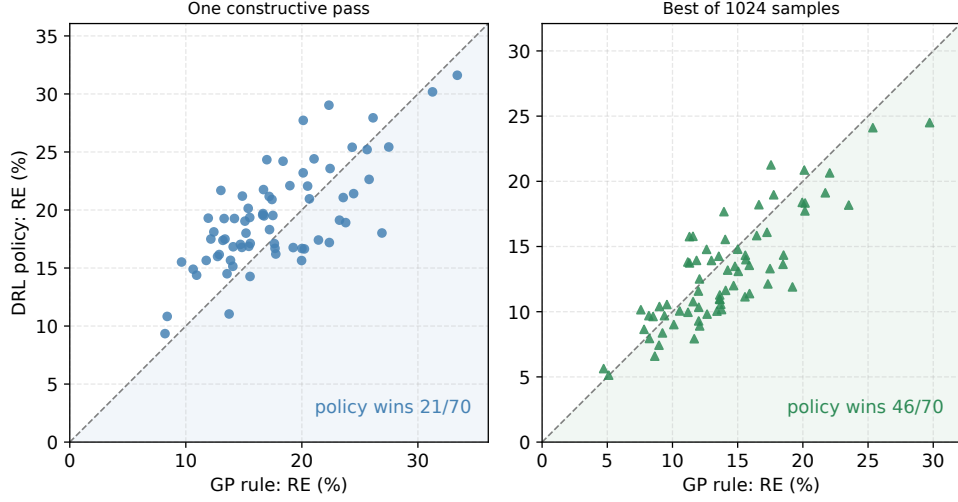


Fig. 3 The DRL policy against the GP rule (Díaz Rodríguez, 2026b) at matched inference budgets, one point per instance over the 70 interval Taillard instances. Points below the diagonal are instances where the policy wins. Left: one constructive pass each. Right: 1024 samples each, the rule randomized with ϵ -greedy dispatching.

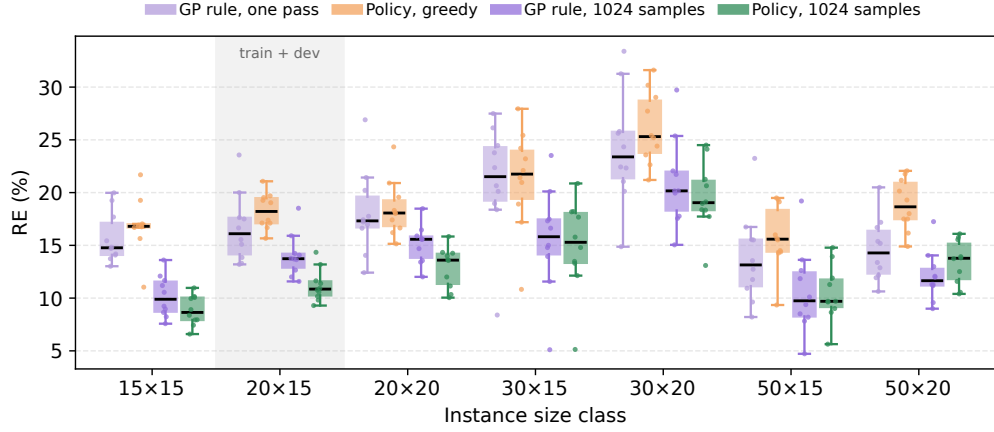


Fig. 4 RE by size class over the 70 interval Taillard instances, the two learners paired at one pass and at 1024 samples. Boxes span the quartiles with whiskers at 1.5 IQR; every instance is overlaid as a point. The shaded 20x15 class holds the training and development instances.

rules (42.3% for EST, the strongest) and the per-instance metaheuristics (9.4% for fEABC). This is the ordering that Zhang et al. (2020) established for the deterministic JSP—a learned constructive policy improves on the hand-crafted rules but does not reach the methods that search each instance anew, trading some solution quality for a cost paid once at training rather than at every instance—and it had not been established for the interval variant.

6.3 Cross-family generalization: the classical instances

The IJSP literature also reports on interval versions of twelve classical instances (FT10 and FT20, seven of the La family, and ABZ7–9), generated from their crisp counterparts with the same symmetric scheme as the Taillard set (Díaz et al., 2020). None of these families was seen during training or development, so they double as a cross-family generalization test. Following the literature, RE is measured here against the best-known expected-makespan values (Díaz et al., 2023b).

Table 9 RE (%) on the 12 classical interval instances. Column groups are inference budgets: GP and the policy (Pol.) are comparable within a group and not across groups. Each entry is the mean over that method’s independent runs, with the per-instance best over those runs in brackets. Reference bounds are best-known expected makespans, not Taillard’s crisp bounds.

Inst.	rule 1 pass	learned one pass		learned 64 samples		learned 1024 samples		search 30 runs
	G&T	GP mn [bst]	Pol. mn [bst]	GP mn [bst]	Pol. mn [bst]	GP mn [bst]	Pol. mn [bst]	GA mn [bst]
FT10	32.2	17.0 [7.2]	16.7 [16.0]	11.8 [5.2]	8.5 [6.3]	8.5 [4.2]	6.1 [5.4]	5.2 [1.8]
FT20	40.0	18.5 [9.4]	8.6 [4.4]	13.3 [5.8]	5.8 [4.4]	10.3 [5.4]	4.4 [3.6]	4.4 [1.5]
La21	24.1	15.9 [10.3]	18.3 [15.5]	12.6 [10.0]	12.6 [10.4]	10.7 [6.8]	10.8 [10.7]	5.0 [3.2]
La24	26.3	16.1 [10.1]	19.3 [17.1]	11.5 [8.6]	11.9 [10.9]	9.2 [6.2]	9.1 [8.1]	6.3 [4.1]
La25	16.9	17.6 [9.6]	17.3 [16.7]	12.3 [8.7]	9.6 [9.2]	9.3 [5.1]	8.9 [8.2]	5.1 [1.9]
La27	34.8	18.3 [12.3]	15.0 [13.8]	12.5 [10.7]	11.3 [10.9]	11.3 [8.2]	9.0 [8.1]	10.2 [4.6]
La29	24.5	19.9 [12.9]	21.1 [16.2]	15.0 [11.4]	16.6 [14.0]	12.6 [9.7]	13.3 [12.9]	14.2 [11.1]
La38	27.0	17.4 [9.2]	17.2 [16.6]	14.3 [9.2]	12.8 [11.7]	12.0 [7.1]	9.6 [8.3]	9.2 [6.0]
La40	27.9	16.7 [11.7]	12.0 [9.3]	12.3 [8.8]	9.3 [8.1]	10.0 [7.1]	7.2 [6.5]	8.7 [5.1]
ABZ7	28.7	17.6 [13.3]	16.7 [15.9]	14.3 [11.7]	13.4 [12.0]	12.2 [8.9]	11.9 [11.1]	12.5 [6.3]
ABZ8	36.9	24.6 [18.1]	21.8 [19.1]	20.9 [17.0]	17.3 [16.9]	18.6 [15.0]	15.8 [15.6]	18.5 [11.3]
ABZ9	41.8	29.0 [19.5]	22.6 [19.7]	24.0 [17.9]	19.2 [18.8]	21.5 [17.4]	17.1 [16.6]	18.0 [13.0]
Mean	30.1	19.0 [12.0]	17.2 [15.0]	14.6 [10.4]	12.4 [11.1]	12.2 [8.4]	10.3 [9.6]	9.8 [5.8]

Table 9 positions the policy (best-of-64, mean and best of the three seeds) against the shared-evaluator rules and the published 30-run averages of the per-instance meta-heuristics, the latter quoted from Table 2 of Díaz et al. (2023b). Its GP columns are the 30 evolutions themselves, the full set being available only on this family, whereas the single featured rule of Section 6.2 represents GP on the Taillard set. The policy improves on EST and G&T-MWKR on all twelve instances and lands at 12.4% mean RE—between the hand-crafted constructive baselines (30.1% for G&T-MWKR) and the per-instance searchers (GA at 9.8%, ESABC at 5.5%). On La29, La40, ABZ7 and ABZ8 the policy’s best seed reaches a lower RE than the 30-run average of GA, a method that searches each of these instances from scratch, whereas the policy has never seen them.

Against the evolved rules the comparison must be made budget by budget, as it was on the Taillard set, and Figure 5 is arranged that way. Given one pass each, the policy’s 17.2% leads the 19.0% a single evolution should be expected to yield, and trails the featured rule’s 17.9% on 5 of the 12 instances (Wilcoxon signed-rank against the featured rule, $p = 0.73$: the two are indistinguishable). Given 64 samples each—the rules randomized by the ϵ -greedy dispatching their own study provides—the policy

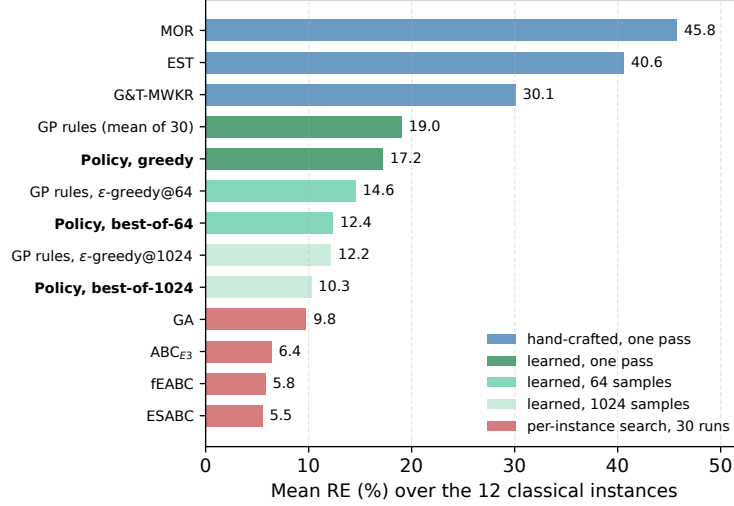


Fig. 5 Mean RE over the 12 classical interval instances, grouped by the inference budget each method is given. Both learned entries are means over independent runs.

leads on both readings, 12.4% against 14.6% for the mean of 30 and 14.2% for the featured rule, better than the latter on 9 of 12, though on twelve instances even that is only marginal ($p = 0.077$). At 1024 samples per run the gap is both the widest and the only one that separates: 10.3% against 12.2%, better on 10 of the 12 instances ($p = 0.009$). Reading these three rows in order, the policy is level with the rules at one pass and pulls away as both are allowed to sample—which is what Section 6.2 finds on the Taillard set, arrived at here on a family neither method was tuned for.

One feature of the table needs explaining: the two readings order the learners oppositely. The policy has the better mean at all three budgets, and GP the better bracket at all three. The cause is the number of runs each bracket selects over—30 evolutions for GP against three training seeds for the policy—compounded by the wider dispersion of the evolved rules: their standard deviation across the 30 averages 4.3, 2.3 and 2.0 points at the three budgets, against 2.2, 1.4 and 0.7 across the policy’s seeds. A per-instance minimum taken from 30 draws of a wide distribution reaches further down than one taken from three draws of a narrow one. Quantitatively, the minimum of n normal draws is expected about 0.85 standard deviations below the mean for $n = 3$ and 2.04 for $n = 30$; selecting the policy over 30 seeds would therefore place its bracket near 12.7%, 9.6% and 8.8%, level with the 12.0%, 10.4% and 8.4% of GP instead of behind them. The bracket ordering measures how many runs each side was selected from, not which learner produces the better run. What is not an artifact is the dispersion across runs itself, smaller for the policy at every budget: evolving one rule and training one network do not expose a practitioner to the same risk from the run they obtain.

7 Analysis

This section reports which inputs the network consults, which of the design choices earn their cost, and how far executed makespans deviate from the predicted values.

7.1 What the policy attends to

The policy is not interpretable the way a priority expression is, but its inputs can be audited. Figure 6 reports a permutation test: at every decision of a greedy rollout, one feature column is shuffled across the eligible operations, and the resulting mean RE over the development set and the three seeds is compared with the intact rollouts (18.2%).

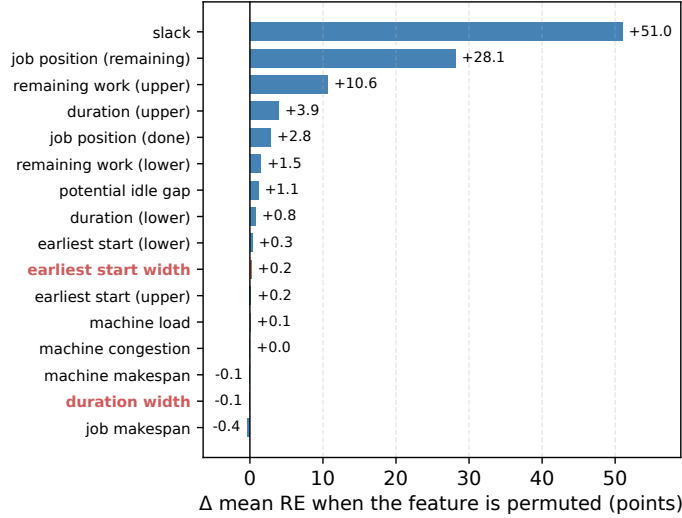


Fig. 6 Permutation importance of the 16 operation features: change in mean development-set RE when the feature’s column is shuffled across the eligible operations at every decision (greedy rollouts, three seeds; intact baseline 18.2%). The two interval-width features are highlighted.

Three features carry the policy: the relative slack (whose permutation adds 51.0 points of RE), the number of remaining operations in the job (+28.1), and the job’s worst-case remaining work (+10.6)—the attribute families behind MOR and MWKR, and the very attributes the companion study’s evolved expressions select (Díaz Rodríguez, 2026b). The interval-width features contribute nothing measurable at inference (−0.1 to +0.2 points), which is what the objective entails rather than a surprise: the worst-case makespan the reward minimizes is a function of the upper endpoints alone (Section 7.5), so a width cannot inform a decision judged by it.

The audit is therefore conditional on what the policy was trained to optimize—this objective and the reward weights of Section 4.1—and a different one need not produce the same ranking. The natural test is an objective that does contain the width, and

Section 7.5 runs it: retraining under f_λ leaves the weights indistinguishable from the default arm’s under a common selection criterion, which is evidence that the ranking would survive that change, though the audit itself was not repeated on those arms.

7.2 Ablation: best-of- N inference

Introducing best-of-64 inference (over one greedy rollout) improved the development-set mean makespan by 3.6%, strictly better on 3 of 6 development instances and worse on none, while sharply reducing seed variance. Diagnostically, before this change the best training solutions frequently arose in early, exploratory episodes and were never surpassed by the greedy policy—evidence that the policy’s value concentrates in its sampling distribution.

The paper reports the policy at three budgets—one greedy pass, 64 samples and 1024—which leaves the shape between them unstated. To recover it we stored all 342 rollouts per checkpoint on each of the 70 instances—71,820 schedules—so that best-of- B can be reconstructed for any B without evaluating anything again. Figure 7 plots the result. Its band is the spread across repetitions at that budget, not a confidence interval on the mean; the budget is split across the three checkpoints as the deployed protocol splits it, and the greedy reference is aggregated the same way, so it sits below the 19.4% of Section 6.2; EST, at 42.3%, is above the frame. A single *sampled* rollout is much worse than a greedy one, 21.4% against 17.0%: the argmax is a good decision sequence and the distribution around it is wide. The curve overtakes the evolved rule at $B=6$ and the greedy pass itself at $B=12$, so the budget at which sampling starts paying is small, far below the 64 used throughout this paper. And the returns are close to logarithmic over three orders of magnitude, 14.9% at 64 and 12.9% at 1023 (341 per checkpoint, the reconstruction’s nearest point to the nominal 1024), with no knee: within the range tested there is no budget beyond which further sampling stops helping.

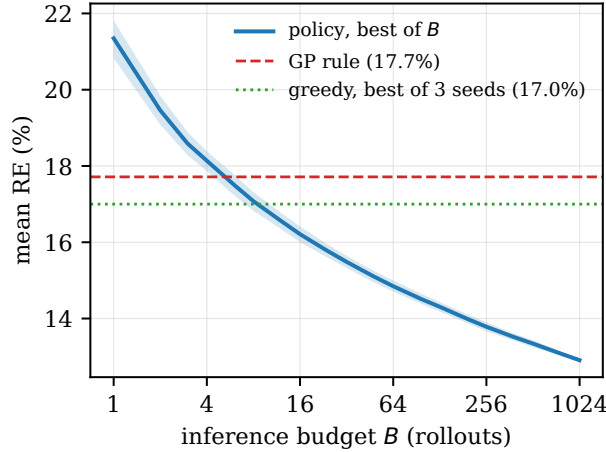


Fig. 7 Mean RE over the 70 instances against the *inference* budget B , the samples spent on each instance, on a base-2 logarithmic axis. The band is the 10–90 percentile over resamplings.

The same reconstruction says how much of that is still worth buying, and at what price: Table 10 gives the mean RE at successive doublings and what each one adds. The return decays slowly but does not vanish: it falls from 1.03 points between 4 and 8 samples to 0.42 between 512 and 1023, a log-linear rate of 0.46 points per doubling over the last three octaves. Cost grows just as fast as the budget. On the machine of Table 4 a sampled rollout takes 1.03 s on a 20×15 instance and 6.4 s on a 50×20 one, so $B=1023$ spends 18 minutes and 1.8 hours per instance, and the four further doublings the measured rate needs to reach 11% would spend 4.7 and 29 hours. That exchange rate argues against closing the remaining distance to fEABC’s 9.4% by sampling alone, which a per-instance metaheuristic covers in minutes. Two caveats bound the arithmetic. Any budget above 1023 lies outside the measured range, and inside it the gain is still shrinking—each octave returns about a tenth less than the one before—so a constant rate is the optimistic reading. And the samples come from a fixed policy, whose distribution bounds what any budget can reach; seeding a search with them (Section 8) carries no such bound.

Table 10 Mean RE over the 70 instances at successive doublings of the inference budget, and what each doubling buys over half its budget. Intermediate budgets, omitted for space, follow the same pattern.

Budget B	1	8	64	128	256	512	1023
Mean RE (%)	21.4	17.1	14.9	14.3	13.8	13.3	12.9
Gain over $B/2$ (points)	—	1.03	0.62	0.57	0.50	0.46	0.42

7.3 Ablation: attention versus pooling

The two-block attention variant of Section 4.4 was evaluated at both operating points with paired seeds: +0.9% mean makespan at 300 episodes (all six instances within the noise threshold) and +1.5% at 1000 (one instance significantly worse), for 3.2 times the parameters and $\approx 30\%$ higher per-episode cost (Table 11). Part of the explanation is in the representation: two of the per-operation features are themselves relational, computed against the eligible set rather than against the candidate alone—slack measures the earliest start relative to the earliest of all candidates, and congestion the machine’s load relative to the average—so the comparisons an attention layer would have to learn are in part precomputed inputs. What remains for it are the pairwise relations those aggregates do not capture, and at these budgets they do not repay their cost; richer pairwise structure may require larger budgets or instance distributions to pay off.

7.4 Ablation: specialist versus multi-size training

Size invariance enables training on mixed size classes, a natural hypothesis for improving transfer. We trained agents on twelve instances spanning 15×15 , 20×15 and 20×20 , at a comparable total budget (3000 episodes, to the specialist’s 4000) and at equalized

Table 11 Attention ablation: mean [best] RE (%) on the development set.

Configuration	300 eps	1000 eps
Deep Sets	16.6 [15.6]	13.4 [12.3]
+ attention ($B=2$)	17.6 [16.3]	15.2 [14.0]

per-instance budget (12,000 episodes—three times the specialist’s total cost), against the 20×15 specialist, all sides with best-of-64 inference and averaged over their three seeds. At comparable total budget the specialist wins on all five common evaluation instances (10.5 vs. 15.9 on 15×15 ; 15.0 vs. 20.0 on 20×20 ; 25.9 vs. 30.6 on the unseen 30×20). Tripling the multi-size budget recovers two of the five (9.6 vs. 10.5 on 15×15 ; 15.8 vs. 16.8 on the unseen 50×20) but not the rest: the specialist stays ahead on 20×20 (15.0 vs. 19.4), a class the multi-size agent itself trained on, and on both 30×15 and 30×20 . At matched cost, per-instance training depth dominates size diversity; even at triple cost, diversity buys back a minority of instances.

Earlier development iterations provide the complementary caution, and two are worth naming. Appending four uncertainty features that are linear combinations of ones already present—the widths and midpoints of quantities the encoder already receives as bounds—cost 9.1% in mean makespan. Normalizing the temporal features by the instance lower bound, an input scaling retrofitted onto an optimizer co-adapted to the raw magnitudes, cost 112.7%: learning collapsed outright. Both figures are against the configuration in force when each was tried, at a reduced screening budget, and both modifications were rejected on that evidence. Representation changes therefore have to be co-designed with the optimization rather than added to a configuration already tuned without them.

7.5 What interval awareness contributes

Componentwise, the upper bound of a completion obeys

$$c_o^U = \max(c_{JP(o)}^U, c_{MP(o)}^U) + p_o^U, \quad (9)$$

a recursion in upper bounds alone: $C_{\max}^U$ is a deterministic function of the processing order and the upper endpoints, and the lower endpoints do not appear in it. The reward of Section 4.1 minimizes exactly that quantity—across its six components not one reads a lower bound—so there is no channel through which knowing an interval’s width could reduce what the policy is being trained to reduce. A width feature can at best be an auxiliary signal for learning; it cannot be information about the objective, because the objective does not contain it.

The first ablation below therefore confirms a property of the objective rather than discovering one. The remaining questions are whether the widths help *learning* even though they cannot help the objective (first), whether the upper bounds seen during training must be the true ones (second, which Eq. (9) does not settle, since training

on midpoints changes the upper bounds themselves), and what happens when the objective is changed so that width does enter it (third).

The representation of Section 4.2 was designed to make uncertainty visible: interval widths enter as relative magnitudes, and the durations, earliest starts and remaining work of a candidate appear as bounds rather than as point estimates. The permutation audit of Section 7.1 already found the two width features unused at inference. Three training-time ablations ask the stronger questions: whether the policy would be any worse had it never been able to see the uncertainty at all, whether it needs the true bounds during training, and whether it would use the widths if the reward paid for them.

The first removes the information. The encoder collapses every interval to its worst case before the features are computed, so the width features are identically zero and the bounds coincide; the architecture, the hyperparameters and the schedule builder are untouched, and the environment still executes interval semantics. Each ablation in this section is paired against the main arm on the same training seeds, so the baseline it subtracts is the mean over the seeds it shares—13.44% for the first three, against the 13.82% of all ten. Three seeds retrained from scratch reach 13.62% mean RE on the development set against that 13.44%—a difference of 0.18 points, worse on twelve of the eighteen instance–seed pairs and better on six, and not significant (Wilcoxon signed-rank, $p = 0.47$). By Eq. (9) this is the expected outcome, and the experiment is worth running only because the prediction concerns the objective while the measurement concerns learning: the widths could in principle have acted as a useful auxiliary signal, shaping the value function or the exploration even while contributing nothing to the quantity minimized. They do not. The effect size makes the bound precise: the paired differences have a standard deviation of 1.54 points, so 0.18 is 0.12 of one, and detecting an effect that small at conventional power would take on the order of ninety seeds. Whatever the widths contribute is therefore an order of magnitude below the run-to-run noise of training itself.

The second removes the uncertainty from training altogether. Because the benchmark perturbs each crisp duration symmetrically, the original crisp instances *are* the midpoint scenario of their interval counterparts, which makes them available as a control: we train on them and evaluate, unchanged, on the interval instances. This one is not covered by Eq. (9)—training on midpoints replaces the upper endpoints themselves, so the policy is fitted to a different problem—and it is the ablation where the extra runs changed the answer. Over three seeds the gap was 0.74 points and not significant; over ten it is 0.63 points, from 13.82% to 14.45%, worse on 36 of the 60 instance–seed pairs and positive for eight of the ten seeds ($p = 0.045$). Training on the true upper bounds is therefore slightly better than training on their midpoints. The margin is small—under five per cent of the RE it improves, at a power of about 0.7, so it should be read as a detected effect and not a measured one—but it is there, and three seeds had not been enough to see it.

Taken together with the permutation audit, the delimitation is sharper than a blanket statement about interval awareness. As *input features* the widths are inert: unused at inference, free to remove, and Eq. (9) says why in a way that generalizes past this network—any method minimizing the worst-case makespan of a semiactive schedule

optimizes a function of the upper endpoints alone, so interval-aware inputs cannot pay for themselves under that objective, whatever the model class. What is *not* inert is the training distribution: the policy fitted to the real upper bounds beats the one fitted to midpoints, which is exactly the case the recursion leaves open. The boundary of the claim is therefore the reward and the data, not the architecture. Under an objective that does contain the width—which has no counterpart in the crisp problem—the companion study finds its width terminals do earn their place (Díaz Rodríguez, 2026b).

The third ablation changes the objective so that the width enters it. We retrain the policy, unchanged in every other respect, under

$$f_\lambda = C_{\max}^U + \lambda (C_{\max}^U - C_{\max}^L), \quad \lambda = 1, \quad (10)$$

which charges the schedule for the width of the interval it predicts and is the analogue of the robust objective of the companion study. Because optimizing one quantity and selecting by another would be incoherent, the best-of- N inference of this arm ranks its samples by f_λ as well. Deployed as that package, the objective does what it promises: over the same eighteen instance–seed pairs the interval narrows from 12.85% to 11.90% of the expected makespan (narrower on 13 of 18, $p = 0.010$) and the expected makespan itself rises by 1.94 points, to 15.38% ($p = 0.007$). Unlike the two ablations above, both effects are significant. A width-rewarding objective is available, and it costs about two points of RE.

Training and selection changed together above, so we separate them: from a fresh pool of 64 rollouts per instance and seed we reconstruct the best-of-64 of *both* arms under one common criterion, so that only the weights differ. Under that control the two policies become indistinguishable—ranking by the upper bound, the widths are 12.80% for the default arm and 12.69% for the robust one ($p = 0.93$); ranking by f_λ , 11.74% and 12.20% ($p = 0.12$), if anything the wrong way round. What narrows the interval is which sample is kept, not which weights produced it: applying the robust criterion to the *default* policy’s own pool already buys most of the reduction, at a comparable cost in RE. The network, in other words, is not merely indifferent to the widths in its input; it does not become sensitive to them when the objective penalizes width.

Three further arms, retrained from scratch at $\lambda = 0.5, 2$ and 4 (three seeds each), show that this is not a quirk of $\lambda = 1$. Deployed as packages—both selections now measured on the fresh rollout pools of the control above, which is why the $\lambda=1$ figures differ slightly from the training-run schedules quoted earlier—the five arms trace a monotone frontier: the relative width decreases with λ —12.80%, 12.38%, 12.20%, 11.88%, 10.99%—while the expected makespan climbs to 17.75% at the far end (+3.49 points over the default arm, $p = 0.0002$). Under the fixed selection the frontier does not appear: no retrained arm proposes narrower rollouts than the default one—three of the four are in fact marginally wider—and no paired width difference comes near significance (smallest $p = 0.37$). The frontier can even be had without the retraining: ranking the default policy’s own pool by each f_λ in turn retraces it almost point for point, ending at 11.09% width and 17.05% RE under f_4 , where the arm actually trained on that objective lands at 10.99% and 17.75%. The predictability–performance trade is therefore real, tunable and monotone, and it arises entirely at selection time.

The companion study runs the same objective at the same λ and reaches the opposite finding, which makes the pair worth reading together. There, penalizing width narrows the schedules only when the rule has width terminals to act through: sweeping λ from 0 to 4 moves the relative width by 1.64 points for rules that have them and by 0.13 points for rules that do not, from which that study concludes that the objective works *through* the representation (Díaz Rodríguez, 2026b). Our network has both the terminals’ equivalent and the objective, and still its weights do not move at any λ ; what moves is the ranking among its samples. The same reward therefore reaches the decision function of an evolved rule and does not reach the decision function of a trained policy, at least at this training scale. The trade itself is priced comparably in the two—the companion pays about 2.5 points of RE per point of width across its sweep, ours about 1.9 across the same range of λ —so the paradigms differ less in what predictability costs than in which part of the system pays it.

Two explanations remain open: that no training signal reaches these features, or that the semiactive constructive setting fixes the width independently of the policy’s decisions.

7.6 Executorial robustness

A schedule is eventually executed under one realization of the durations, and its interval prediction is useful insofar as the executed makespan stays close to the reported expected value. We adopt the field’s $\bar{\varepsilon}$ measure (Díaz et al., 2020; Leon, Wu, & Storer, 1994): for a fixed processing order with predicted makespan $\mathbf{C}_{\max}$,

$$\bar{\varepsilon} = \frac{1}{K} \sum_{k=1}^K \frac{|C_{\max}^{\text{ex},k} - E[\mathbf{C}_{\max}]|}{E[\mathbf{C}_{\max}]}, \quad (11)$$

where $C_{\max}^{\text{ex},k}$ is the crisp makespan of executing the order under the k -th realization, durations drawn independently and uniformly within their intervals. We use $K=1000$ common random numbers per instance—identical realizations for every method, drawn from a generator seeded by the instance identifier so that the draws reproduce across runs and machines—and report $\bar{\varepsilon} \times 10^3$. The Monte Carlo is the expensive part, so this section uses fifteen instances rather than the full seventy: the six of the development set and the nine spotlight instances of the per-instance cross-size evaluations of Section 6.2 (TA1, TA2, TA5, TA21, TA25, TA31, TA41, TA51, TA61), reused here unchanged so that no instance is selected with this comparison in view. Every execution falls inside the predicted interval, as the monotonicity of Eq. (1) guarantees.

The constructive methods separate into two groups, and the policy is in the better one without leading it (Figure 8). Its schedules deviate by $\bar{\varepsilon} \times 10^3 = 6.18$ under greedy inference and 6.12 under best-of-64—sampling for a lower worst case neither helps nor harms predictability—against 6.86 for G&T-MWKR and 7.14 for MOR, and those two gaps are systematic (the policy is more faithful on 13/15 and 14/15 instances respectively, paired Wilcoxon signed-rank test, $p < 0.001$ in both cases). Against the other two the policy is indistinguishable: EST deviates by 6.35 (9/15, $p = 0.30$) and the evolved rule of the companion study by 6.12 (9/15, $p = 0.72$), level with the policy and well inside the noise.

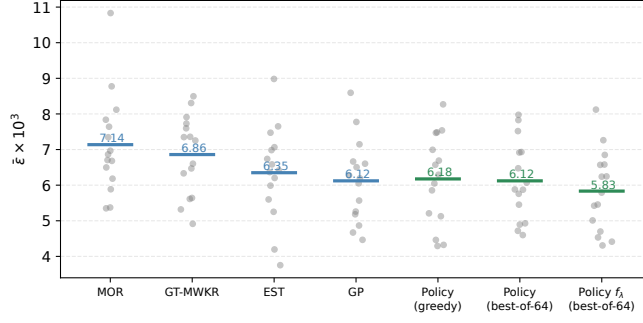


Fig. 8 Executional robustness by method: mean $\bar{\varepsilon} \times 10^3$ (bars) over the fifteen evaluation instances, each shown as a point. Lower is a more faithful a-priori prediction. EST, the evolved rule and the policy are indistinguishable at the top and MOR and G&T-MWKR are not; the robust arm f_λ , the only method here that pays for the interval width, is ahead of all of them but not significantly so.

Four methods therefore share the top of this ranking—a hand-crafted rule, an evolved formula and the policy under two inference budgets—and they have one thing in common: none of them optimizes the width of the interval it predicts. A schedule can deviate little without being short, so faithfulness has to be pursued deliberately rather than inherited from a makespan objective; the companion study reaches the same conclusion from its side, where its rule likewise improves on G&T-MWKR but not on EST until an objective that penalizes width is introduced (Díaz Rodríguez, 2026b).

The robust arm of Section 7.5 is the direct test of that reading, since it is the one method here that does charge for the width, and it comes out on top: $\bar{\varepsilon} \times 10^3 = 5.83$ with the narrowest predictions of the pool (11.5% of the expected makespan against 12.3% for the default arm). The improvement over the default policy is not significant on fifteen instances (-0.29 , more faithful on 9/15, $p = 0.23$), so this is a direction and not a demonstration. But it is the direction the argument predicts, and it comes at the price Section 7.5 measured: about two points of expected makespan. Whether that trade is worth making depends on the shop rather than on the model; this section’s earlier comparison shows that the trade is obtained only when the objective charges for the width.

7.7 Limitations

Four limitations bound the scope of these conclusions. First, the benchmark follows the symmetric $\pm 15\%$ generation scheme standard in the IJSP literature; asymmetric intervals, and distributions in which the uncertainty varies sharply from one operation to another, would separate the worst-case and expected-value criteria far more than they do here, and no policy was trained under them. Second, the policy builds semiactive schedules, so the space it searches excludes the active schedules that the conflict-set restriction reaches and the insertion-based ones beyond them; the strongest constructive baseline of this paper exploits the first of those and the policy still leads it, but the ceiling is real. Third, the ablations are run over three training seeds and

compared pairwise on them; ten seeds of the main arm bound how much that choice can matter (Section 6.1), but the ablation arms themselves were not extended, so their differences are stated as paired effects over eighteen instance–seed pairs rather than as claims about the population of seeds. Finally, the comparison against the evolved rule of Section 6.2 matches inference budgets but not training ones, and it compares two published artifacts rather than two paradigms trained under one protocol; that study, which is what Xu et al. (2025) call for, and the use of the policy’s sampling distribution to seed population-based search, are the subject of dedicated ongoing work.

8 Conclusions and Future Work

The three questions posed in the introduction now have answers. A constructive policy trained by PPO on four interval instances beats the best hand-crafted constructive baselines by a wide margin on its training class (Q1: 13.8% versus 27.9% for G&T-MWKR and $\approx 46\%$ for the best plain rule). Because the representation is dimensionless and the encoder permutation-invariant, one network serves the entire benchmark: evaluated zero-shot, it outperforms the best dispatching rule on every tested instance of all seven size classes, and the same weights improve on every rule on all twelve classical instances of the FT, La and ABZ families (Q2). Against a rule evolved by genetic programming the answer is not one-sided but budget-dependent: pass for pass the rule is better, and the policy overtakes it only when both are given 1024 samples (Section 6.2), so what the network contributes over a compact formula is its sampling distribution rather than its argmax. Of the standard techniques evaluated, only best-of- N inference improved results at practical budgets: self-attention did not improve on pooling, multi-size training did not overtake the single-size specialist even at three times its cost, and two independent automatic-configuration campaigns failed to beat the untuned defaults at the operating budget (Q3). Three more delimit the interval representation that motivated the design, and they separate two things the design had conflated. As inputs the widths are inert: the policy neither uses them at inference (-0.1 points when they are shuffled) nor suffers when they are removed from training (13.62 against 13.44%, $p = 0.47$), which the closed recursion for the worst-case makespan predicts, since that quantity does not contain a lower bound. As training data the intervals are not inert: fitting the policy to the crisp midpoints instead costs 0.63 points over ten seeds (14.45 against 13.82%, $p = 0.045$). Retraining under an objective that charges for the width does not overturn this delimitation: swept from $\lambda = 0$ to 4, the deployed arms trace a monotone width–makespan frontier (the predicted interval narrows from 12.80% to 10.99% of the expected makespan, at about 1.9 points of RE per point of width), but with the selection criterion held fixed the robust weights and the default ones are indistinguishable, and ranking the default policy’s own samples by f_λ retraces the frontier without any retraining—what narrows the interval is which sample is kept, not which network produced it. At execution time the policy’s schedules are the most dependable of the constructive pool, matching EST, the robustness benchmark that evolved rules also fail to beat; and a permutation audit shows the network leaning on slack, remaining operations and worst-case remaining

work—the attributes hand-crafted and evolved rules exploit—while ignoring the width features at inference.

Three directions follow. The first is suggested by the ablation of Section 7.4: since a specialist transfers so well and multi-size training buys so little, the productive use of size invariance is probably not to train on many sizes at once but to fine-tune a specialist onto a new class cheaply, a curriculum question this paper leaves open. The second concerns the representation, and the robust arm of Section 7.5 sharpens rather than settles it: paying for the width in the reward narrows the schedules the policy *selects* without measurably changing the policy itself, so the width features remain unconsulted even when the objective would reward consulting them. Whether the gradient simply never reaches them, or whether a semiactive constructive decision determines the width before the policy has any say in it, separates two very different diagnoses and calls for a different experiment than any run here—a reward defined on the width alone would tell them apart. The third is deployment: a policy that schedules an instance in seconds is a natural generator of initial solutions for population-based search, and the quality of its sampling distribution, documented here at a fixed budget, is what such a use would exploit.

Appendix A Per-instance results

Table A1 lists, for each of the 70 interval Taillard instances, the crisp lower bound and the RE (%) of the earliest-start rule, of Giffler–Thompson with MWKR tie-breaking, of the evolved rule of the companion study (Díaz Rodríguez, 2026b), and of the policy at its two inference budgets: greedy, averaged over the three training seeds, and best-of-1024. The ten 20×15 instances TA11–TA20 are the training and development set.

Statements and Declarations

Funding.

This work was supported by the Spanish Ministry of Science, Innovation and Universities (MCIN/AEI/10.13039/501100011033) under grant PID2022-141746OB-I00.

Competing interests.

The author has no competing interests to declare that are relevant to the content of this article.

Data availability.

The benchmark instances are openly available in a Zenodo repository (Díaz Rodríguez, 2026a). Code, benchmark harness, all experiment records (accepted and rejected), final checkpoints and figure-generation scripts will be deposited openly upon submission. [TODO: DOI del depósito propio al enviar]

Table A1 Per-instance RE (%) of the constructive methods and of the policy. LB is the crisp Taillard lower bound; Pol. is the greedy policy and Pol.* the best-of-1024 policy.

Inst.	LB	EST	G&T	GP	Pol.	Pol.*	Inst.	LB	EST	G&T	GP	Pol.	Pol.*
TA1	1231	49.9	26.1	19.3	16.8	11.0	TA36	1819	41.1	42.8	23.8	18.9	13.3
TA2	1244	24.9	27.8	14.2	19.3	6.6	TA37	1771	43.4	35.6	18.4	24.2	17.7
TA3	1218	33.0	33.2	14.1	16.8	10.1	TA38	1673	45.0	28.0	20.1	23.2	14.8
TA4	1175	32.0	27.7	13.0	21.7	10.0	TA39	1795	46.6	28.9	22.4	17.2	13.5
TA5	1224	27.2	33.0	14.7	17.0	8.4	TA40	1651	46.9	42.3	26.1	27.9	20.9
TA6	1238	26.8	18.2	13.7	11.0	7.4	TA41	1906	44.6	45.0	33.4	31.6	24.5
TA7	1227	30.7	26.1	17.7	16.7	7.9	TA42	1884	71.1	38.1	24.3	25.4	18.3
TA8	1217	23.7	25.0	14.8	16.8	7.9	TA43	1809	68.7	39.8	22.3	29.0	20.6
TA9	1274	37.7	28.2	20.0	15.6	8.9	TA44	1948	52.1	34.7	25.8	22.6	19.0
TA10	1241	37.0	22.1	15.4	16.9	10.2	TA45	1997	50.2	33.7	14.9	21.2	13.1
TA11	1357	56.2	30.7	17.5	19.5	10.6	TA46	1957	60.8	31.6	22.4	23.6	17.7
TA12	1367	48.8	34.6	17.6	17.1	10.9	TA47	1807	46.8	30.1	25.6	25.2	19.1
TA13	1342	60.2	26.9	13.3	19.3	13.2	TA48	1912	59.1	33.7	21.1	24.4	18.4
TA14	1345	39.5	29.7	13.2	17.4	9.3	TA49	1931	51.0	35.3	20.1	27.7	21.3
TA15	1339	48.1	30.6	16.7	19.7	11.4	TA50	1833	61.4	46.2	31.3	30.2	24.1
TA16	1360	45.2	34.0	15.6	17.1	10.8	TA51	2760	33.8	34.7	23.2	19.1	11.9
TA17	1462	58.9	17.7	13.9	15.7	9.8	TA52	2756	38.0	26.5	12.7	16.0	11.3
TA18	1377	43.8	31.8	23.6	21.1	14.3	TA53	2717	34.1	18.3	11.8	15.7	9.7
TA19	1332	45.8	29.3	20.0	16.7	11.6	TA54	2839	20.9	16.1	8.2	9.3	5.6
TA20	1348	45.7	24.1	15.1	19.0	10.2	TA55	2679	31.7	29.2	15.5	19.4	14.8
TA21	1642	33.4	28.0	20.2	16.6	11.1	TA56	2781	34.8	19.8	10.9	14.4	8.6
TA22	1561	44.2	30.8	17.4	20.9	14.3	TA57	2943	32.9	20.0	9.6	15.5	9.6
TA23	1518	42.8	35.3	17.8	16.2	13.6	TA58	2885	37.4	26.3	13.6	14.5	9.0
TA24	1644	56.3	23.8	14.1	15.1	10.3	TA59	2655	26.6	28.2	16.7	19.5	13.9
TA25	1558	45.8	33.8	16.6	19.6	14.0	TA60	2723	41.7	19.8	15.5	14.3	9.7
TA26	1591	42.8	31.2	26.9	18.0	14.3	TA61	2868	45.7	25.4	15.4	20.1	13.9
TA27	1652	50.0	31.1	21.4	17.4	13.6	TA62	2869	43.0	27.6	17.2	21.2	15.5
TA28	1603	36.8	26.8	12.4	18.1	10.0	TA63	2755	38.8	23.8	13.4	17.5	13.7
TA29	1583	28.8	32.0	17.2	18.3	12.0	TA64	2702	38.2	28.4	12.2	17.5	11.6
TA30	1528	34.2	36.1	17.0	24.3	15.8	TA65	2725	42.9	30.0	20.5	22.1	16.1
TA31	1764	30.7	34.4	24.5	21.4	12.1	TA66	2845	41.8	28.0	11.9	19.3	13.8
TA32	1774	42.7	33.3	19.0	22.1	18.2	TA67	2825	31.7	26.1	15.2	18.0	12.5
TA33	1788	51.3	34.8	27.5	25.4	18.2	TA68	2784	50.1	19.1	10.6	14.9	10.4
TA34	1828	45.9	31.8	20.7	21.0	15.8	TA69	3071	35.3	22.8	12.9	16.2	10.6
TA35	2007	30.3	24.1	8.4	10.8	5.1	TA70	2995	42.9	23.4	16.7	21.8	15.8

References

- Bello, I., Pham, H., Le, Q.V., Norouzi, M., Bengio, S. (2016). *Neural combinatorial optimization with reinforcement learning*. (arXiv:1611.09940)
- Branke, J., Nguyen, S., Pickardt, C.W., Zhang, M. (2016). Automated design of production scheduling heuristics: a review. *IEEE Transactions on Evolutionary Computation*, 20(1), 110–124, <https://doi.org/10.1109/tevc.2015.2429314>
- Díaz, H., González-Rodríguez, I., Palacios, J.J., Díaz, I., Vela, C.R. (2020). A genetic approach to the job shop scheduling problem with interval uncertainty. *Information processing and management of uncertainty in knowledge-based systems (IPMU)* (pp. 663–676). Springer.

- Díaz, H., Palacios, J.J., Díaz, I., Vela, C.R., González-Rodríguez, I. (2022). Robust schedules for tardiness optimization in job shop with interval uncertainty. *Logic Journal of the IGPL*, 31(2), 240–254, <https://doi.org/10.1093/jigpal/jzac016>
- Díaz, H., Palacios, J.J., González-Rodríguez, I., Vela, C.R. (2023a). An elitist seasonal artificial bee colony algorithm for the interval job shop. *Integrated Computer-Aided Engineering*, 30(3), 223–242, <https://doi.org/10.3233/ica-230705>
- Díaz, H., Palacios, J.J., González-Rodríguez, I., Vela, C.R. (2023b). Fast elitist ABC for makespan optimisation in interval JSP. *Natural Computing*, 22(4), 645–657, <https://doi.org/10.1007/s11047-023-09953-2>
- Díaz Rodríguez, H. (2026a). *Genetic programming dispatching rules for the interval job shop: instances, evolved rules, results and code*. Zenodo. ([dataset])
- Díaz Rodríguez, H. (2026b). *Genetic programming hyper-heuristics for the job shop scheduling problem with interval durations: robustness-aware interpretable rules that generalize across instance sizes*. (Preprint. [TODO: identificador del preprint])
- Garey, M.R., Johnson, D.S., Sethi, R. (1976). The complexity of flowshop and jobshop scheduling. *Mathematics of Operations Research*, 1(2), 117–129, <https://doi.org/10.1287/moor.1.2.117>
- Giffler, B., & Thompson, G.L. (1960). Algorithms for solving production-scheduling problems. *Operations Research*, 8(4), 487–503, <https://doi.org/10.1287/opre.8.4.487>
- González-Rodríguez, I., Vela, C.R., Puente, J. (2010). A genetic solution based on lexicographical goal programming for a multiobjective job shop with uncertainty. *Journal of Intelligent Manufacturing*, 21(1), 65–73, <https://doi.org/10.1007/s10845-008-0161-x>
- Haupt, R. (1989). A survey of priority rule-based scheduling. *OR Spektrum*, 11(1), 3–16, <https://doi.org/10.1007/bf01721162>
- Kool, W., van Hoof, H., Welling, M. (2019). Attention, learn to solve routing problems! *International conference on learning representations (ICLR)*.

- Lei, D. (2011). Population-based neighborhood search for job shop scheduling with interval processing time. *Computers & Industrial Engineering*, 61(4), 1200–1208, <https://doi.org/10.1016/j.cie.2011.07.010>
- Leon, V.J., Wu, S.D., Storer, R.H. (1994). Robustness measures and robust scheduling for job shops. *IIE Transactions*, 26(5), 32–43, <https://doi.org/10.1080/07408179408966626>
- López-Ibáñez, M., Dubois-Lacoste, J., Pérez Cáceres, L., Birattari, M., Stützle, T. (2016). The irace package: Iterated racing for automatic algorithm configuration. *Operations Research Perspectives*, 3, 43–58, <https://doi.org/10.1016/j.orp.2016.09.002>
- Nazari, M., Oroojlooy, A., Snyder, L., Takáč, M. (2018). Reinforcement learning for solving the vehicle routing problem. *Advances in neural information processing systems (NeurIPS)*.
- Nguyen, S., Mei, Y., Zhang, M. (2017). Genetic programming for production scheduling: a survey with a unified framework. *Complex & Intelligent Systems*, 3(1), 41–66, <https://doi.org/10.1007/s40747-017-0036-x>
- Palacios, J.J., González-Rodríguez, I., Vela, C.R., Puente, J. (2015). Coevolutionary makespan optimisation through different ranking methods for the fuzzy flexible job shop. *Fuzzy Sets and Systems*, 278, 81–97, <https://doi.org/10.1016/j.fss.2014.12.003>
- Panwalkar, S.S., & Iskander, W. (1977). A survey of scheduling rules. *Operations Research*, 25(1), 45–61, <https://doi.org/10.1287/opre.25.1.45>
- Park, J., Chun, J., Kim, S.H., Kim, Y., Park, J. (2021). Learning to schedule job-shop problems: representation and policy learning using graph neural network and reinforcement learning. *International Journal of Production Research*, 59(11), 3360–3377, <https://doi.org/10.1080/00207543.2020.1870013>
- Schulman, J., Wolski, F., Dhariwal, P., Radford, A., Klimov, O. (2017). *Proximal policy optimization algorithms*. (arXiv:1707.06347)
- Sotskov, Y.N., Matsveichuk, N.M., Hatsura, V.D. (2020). Schedule execution for two-machine job-shop to minimize makespan with uncertain processing times. *Mathematics*, 8(8), 1314, <https://doi.org/10.3390/math8081314>

- Taillard, E. (1993). Benchmarks for basic scheduling problems. *European Journal of Operational Research*, 64(2), 278–285, [https://doi.org/10.1016/0377-2217\(93\)90182-M](https://doi.org/10.1016/0377-2217(93)90182-M)
- Tassel, P., Gebser, M., Schekotihin, K. (2021). *A reinforcement learning environment for job-shop scheduling*. (arXiv:2104.03760)
- Đurašević, M., & Jakobović, D. (2018). A survey of dispatching rules for the dynamic unrelated machines environment. *Expert Systems with Applications*, 113, 555–569, <https://doi.org/10.1016/j.eswa.2018.06.053>
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., . . . Polosukhin, I. (2017). Attention is all you need. *Advances in neural information processing systems (NeurIPS)*.
- Vinyals, O., Fortunato, M., Jaitly, N. (2015). Pointer networks. *Advances in neural information processing systems (NeurIPS)*.
- Xiong, R., Yang, Y., He, D., Zheng, K., Zheng, S., Xing, C., . . . Liu, T.-Y. (2020). On layer normalization in the transformer architecture. *International conference on machine learning (ICML)*.
- Xu, M., Mei, Y., Zhang, F., Zhang, M. (2025). Learn to optimise for job shop scheduling: a survey with comparison between genetic programming and reinforcement learning. *Artificial Intelligence Review*, 58, 160, <https://doi.org/10.1007/s10462-024-11059-9>
- Zaheer, M., Kottur, S., Ravanbakhsh, S., Póczos, B., Salakhutdinov, R., Smola, A. (2017). Deep sets. *Advances in neural information processing systems (NeurIPS)*.
- Zhang, C., Song, W., Cao, Z., Zhang, J., Tan, P.S., Xu, C. (2020). Learning to dispatch for job shop scheduling via deep reinforcement learning. *Advances in neural information processing systems (NeurIPS)*.
- Zheng, P., Xiao, S., Zhang, P., Lv, Y. (2024). A two-individual-based evolutionary algorithm for flexible assembly job shop scheduling problem with uncertain interval processing times. *Applied Sciences*, 14(22), 10304, <https://doi.org/10.3390/app142210304>